미들웨어 구현과 코드 읽기 (실습)

2-1. 로깅 미들웨어 구현 (실습)
2-2. 요약, 마스킹 미들웨어 (실습)
2-3. 미들웨어 조합 (실습)

2-1. 로깅 미들웨어 구현 (실습)

> 모든 요청과 응답을 로컬 파일 agent_log.jsonl에 기록하는 미들웨어를 직접 구현해봅니다.

가장 단순하면서 실무 활용도가 높은 미들웨어입니다. 모델 호출 직전에 입력을 기록하고 직후에 응답과 도구 호출 결정을 기록합니다. 운영 환경에서 "Agent가 왜 이런 답을 했지?"를 추적하는 첫 번째 수단이 됩니다.

기록 형식은 JSON Lines(jsonl)를 씁니다. 한 줄에 JSON 객체 하나를 적는 형식입니다. 파일 끝에 한 줄씩 계속 덧붙일 수 있고 한 줄씩 읽어 그대로 분석할 수 있습니다. 그래서 로그 표준으로 널리 쓰입니다.

여기서는 완성 코드를 보지 않고 직접 타이핑해 완성하는 것이 목표입니다. 1번째 세션의 훅 표를 보면서 어느 훅에 무엇을 기록할지 먼저 정하고 코드로 옮깁니다.

따라하기

1. 로깅 미들웨어를 작성해 `day5_practice/logging_middleware.py` 로 저장합니다.

```
# logging_middleware.py - 모든 요청과 응답을 agent_log.jsonl에 기록
import json
import datetime
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage

load_dotenv()


@tool
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다."""
    # 교육용 더미 데이터입니다
    fake_db = {
        "김하늘": "김하늘 / 클라우드운영팀 / haneul.kim@samsungsds.example.com",
        "박도윤": "박도윤 / 물류플랫폼팀 / doyun.park@samsungsds.example.com",
    }
    return fake_db.get(name, f"{name}을(를) 찾을 수 없습니다.")


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


class LoggingMiddleware(AgentMiddleware):
    """모델 호출 전후를 agent_log.jsonl 파일에 기록하는 미들웨어"""

    def _write(self, record: dict) -> None:
        record["time"] = datetime.datetime.now().isoformat()
        with open("agent_log.jsonl", "a", encoding="utf-8") as f:
            f.write(json.dumps(record, ensure_ascii=False) + "\n")

    def before_model(self, state, runtime):
        """모델 호출 전: 마지막 입력 메시지를 기록합니다."""
        last = state["messages"][-1] if state["messages"] else None
        if last:
            self._write({
```

```
                "event": "before_model",
                "message_type": type(last).__name__,
                "preview": get_text(last)[:80],
            })
        return None  # State 변경 없음

    def after_model(self, state, runtime):
        """모델 호출 후: 응답 타입, 도구 호출 결정, 토큰 사용량을 기록합니다."""
        last = state["messages"][-1]
        record = {"event": "after_model", "message_type": type(last).__name_
_}
        if hasattr(last, "tool_calls") and last.tool_calls:
            record["tool_calls"] = [t["name"] for t in last.tool_calls]
        if hasattr(last, "usage_metadata") and last.usage_metadata:
            record["total_tokens"] = last.usage_metadata.get("total_tokens",
0)
        self._write(record)
        return None


if __name__ == "__main__":
    llm = ChatBedrockConverse(
        model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
        region_name="us-east-1",
    )
    agent = create_agent(
        model=llm,
        tools=[search_employee],
        middleware=[LoggingMiddleware()],
    )
    result = agent.invoke({"messages": [HumanMessage("김하늘 소속 팀 알려줘")]})
    print("최종 응답:", get_text(result["messages"][-1]))
```

2. 실행 후 같은 폴더에 생긴 `agent_log.jsonl` 을 편집기로 열어 기록을 확인합니다.

```
{"event": "before_model", "message_type": "HumanMessage", "preview": "김하늘
소속 팀 알려줘", "time": "2026-08-13T10:02:11.482910"}
{"event": "after_model", "message_type": "AIMessage", "tool_calls": ["search
_employee"], "total_tokens": 412, "time": "2026-08-13T10:02:13.105331"}
{"event": "before_model", "message_type": "ToolMessage", "preview": "김하늘 /
클라우드운영팀 / haneul.kim@samsungsds.example.com", "time": "2026-08-13T10:02:1
3.121904"}
{"event": "after_model", "message_type": "AIMessage", "total_tokens": 655,
"time": "2026-08-13T10:02:14.870122"}
```

3. 질문을 도구가 필요 없는 것으로 바꿔 다시 실행하고 기록이 두 줄만 늘어나는 것을 비교합니다.

막힐 때

- `state["messages"][-1]` 에서 `IndexError` 가 납니다.

  메시지가 비어 있는 경우입니다. 위 코드처럼 `if state["messages"]` 체크를 먼저 하는 습관을 들입니다.

- `usage_metadata` 가 항상 있는 것은 아닙니다.

  반드시 `hasattr` 체크 후에 접근합니다. Bedrock에서도 LangChain이 형식을 통일해 주므로 같은 코드가 동작합니다.

- 로그 파일의 한글이 깨져 보입니다.

  `json.dumps` 에 `ensure_ascii=False` 를 빼먹은 경우입니다.

- 훅 안에서 예외가 나면 Agent 전체 호출이 실패합니다.

  로깅처럼 부가적인 미들웨어는 실패해도 본 흐름을 막지 않도록 `try` / `except` 로 감싸는 정책을 고려합니다.

확인: `agent_log.jsonl` 에 before_model과 after_model 기록이 시간순으로 쌓이고 도구를 쓴 호출에는 `tool_calls` 항목이 보이면 성공입니다.

> 모든 요청과 응답을 파일 기록으로 남기는 관측의 첫 수단을 직접 만들었습니다.

2-2. 요약, 마스킹 미들웨어 (실습)

> 완성 코드를 읽고 실행하며 요약과 마스킹이 각각 어느 훅에서 왜 동작하는지 확인해봅니다.

여기서는 직접 구현이 아니라 코드 리딩이 중심입니다. 완성 코드를 실행해 보고 왜 그 훅에서 처리하는지를 읽어내는 것이 목표입니다.

먼저 요약 미들웨어입니다. 대화가 길어지면 오래된 메시지를 LLM으로 요약해 압축하고 최근 메시지만 원본으로 유지합니다.

이렇게 하면 토큰 비용을 줄이고 컨텍스트 윈도우 초과를 막습니다. 직접 짤 필요 없이 내장 `SummarizationMiddleware` 를 씁니다.

따라하기

1. 요약 미들웨어 완성 코드를 `day5_practice/summarization_demo.py` 로 저장해 실행합니다. 실습에서는 임계값을 낮게 잡아 요약을 빨리 관찰합니다.

```
# summarization_demo.py - 내장 요약 미들웨어 관찰 (코드 리딩)
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware
from langchain_aws import ChatBedrockConverse
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages import HumanMessage

load_dotenv()

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

agent = create_agent(
    model=llm,
    tools=[],
    middleware=[
        SummarizationMiddleware(
            model=llm,
            max_tokens_before_summary=500,   # 실습용으로 낮게 잡아 요약을 빨리 관찰
            messages_to_keep=4,              # 최근 4개만 원본 유지
        ),
    ],
    checkpointer=InMemorySaver(),  # 멀티턴이어야 요약을 관찰할 수 있습니다 (3-1 참고)
)

config = {"configurable": {"thread_id": "summary_demo"}}
questions = [
    "사내 어시스턴트가 뭘 할 수 있는지 세 문장으로 설명해줘",
    "LangChain의 장점을 다섯 문장으로 설명해줘",
    "LangGraph와의 차이를 다섯 문장으로 설명해줘",
    "지금까지 대화를 바탕으로 오늘 배울 주제를 추천해줘",
]
for q in questions:
    result = agent.invoke({"messages": [HumanMessage(q)]}, config=config)
    print(f"질문: {q}")
    print(f"  현재 메시지 개수: {len(result['messages'])}")
```

2. 같은 thread_id로 대화를 이어가면 어느 순간 메시지 개수가 줄고 맨 앞에 요약 메시지가 생기는 것을 확인합니다. 읽기 포인트는 하나입니다. 요약은 모델을 부르기 전에 입력을 줄이는 일이므로 `before_model` 계열 시점에 동작합니다.

3. 다음은 마스킹 미들웨어입니다. 입력에 섞인 개인정보(PII, Personally Identifiable Information)를 모델에 보내기 전에 가립니다. PII는 개인을 특정할 수 있는 정보라는 뜻이고 여기서는 사번, 이메일, 전화번호를 다룹니다.
   완성 코드를 `day5_practice/masking_middleware.py`로 저장해 실행하고 아래 읽기 포인트를 따라 읽습니다.

```
# masking_middleware.py - 개인정보 마스킹 미들웨어 (코드 리딩)
import re
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage

load_dotenv()

# 마스킹 대상: 사번, 이메일, 전화번호 (교육용 더미 데이터 기준)
PII_PATTERNS = {
    "emp_id": r"\b\d{8}\b",                                          # 사번 (8자
리 숫자)
    "phone": r"01[0-9][-\s]?\d{3,4}[-\s]?\d{4}",                     # 휴대전화
    "email": r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",     # 이메일
}


def mask_pii(text: str) -> str:
    for kind, pattern in PII_PATTERNS.items():
        text = re.sub(pattern, f"[MASKED_{kind.upper()}]", text)
    return text


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


def mask_message_content(message):
    """읽기가 아니라 고쳐서 되돌려주는 자리라 블록 구조를 살려 둔 채 텍스트만 마스킹합니다."""
    content = message.content
    if isinstance(content, list):
        return [
            {**block, "text": mask_pii(block["text"])}
            if isinstance(block, dict) and "text" in block else block
            for block in content
        ]
    return mask_pii(content)
```

```
class MaskingMiddleware(AgentMiddleware):
    """모델 호출 전에 사용자 입력의 개인정보를 마스킹하는 미들웨어"""

    def before_model(self, state, runtime):
        # 가장 최근 사용자 메시지를 찾습니다
        last_human = next(
            (m for m in reversed(state["messages"]) if isinstance(m, HumanMe
ssage)),
            None,
        )
        if not last_human:
            return None

        masked = mask_message_content(last_human)
        if masked != last_human.content:
            print("[masking] 개인정보 마스킹 적용")
            # 찾은 사용자 메시지를 마스킹된 버전으로 교체합니다
            # 같은 id로 돌려주면 add_messages가 그 메시지를 제자리에서 교체합니다
            return {"messages": [HumanMessage(content=masked, id=last_human.
id)]}
        return None


if __name__ == "__main__":
    llm = ChatBedrockConverse(
        model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
        region_name="us-east-1",
    )
    agent = create_agent(model=llm, tools=[], middleware=[MaskingMiddleware
()])
    # 출력할 때도 content가 블록 리스트일 수 있으므로 get_text를 거칩니다
    result = agent.invoke({"messages": [HumanMessage(
        "김하늘 사번이 25010042 맞는지 확인해줘. 연락처는 010-2345-6789이고 "
        "메일은 haneul.kim@samsungsds.example.com이야."
    )]})
    print("최종 응답:", get_text(result["messages"][-1]))
```

실행 결과 예시

```
[masking] 개인정보 마스킹 적용
모델이 실제로 받은 입력: 김하늘 사번이 [MASKED_EMP_ID] 맞는지 확인해줘. 연락처는
[MASKED_PHONE]이고 메일은 [MASKED_EMAIL]이야.
```

읽기 포인트는 세 가지입니다.

- 왜 `before_model`인가: 개인정보가 외부 모델에 전송되기 전에 가려야 의미가 있기 때문입니다. `after_model`에서 가리면 이미 전송된 뒤입니다.
- 왜 `reversed`로 찾는가: 도구를 쓰는 Agent는 메시지 끝이 `ToolMessage`일 수 있어서 가장 최근의 `HumanMessage`를 뒤에서부터 찾아야 합니다.
- 왜 `id=last_human.id`를 실어 보내는가: `messages`는 add_messages 규칙으로 병합되므로 새 id로 보내면 교체가 아니라 추가가 됩니다. 같은 id를 실어야 그 메시지가 제자리에서 마스킹된 내용으로 바뀝니다.

막힐 때

- 요약이 관찰되지 않습니다.

  임계값(`max_tokens_before_summary`)을 더 낮추거나 대화를 더 길게 이어갑니다. thread_id가 매번 달라지면 대화가 누적되지 않아 요약이 시작되지 않습니다.

- 정규식은 사번, 전화번호, 이메일처럼 패턴이 명확한 것만 잡습니다.

  이름과 주소 같은 것은 못 잡으므로 필요하면 LLM 판별을 병행합니다. 4번째 세션 4-2에서 같은 전략을 씁니다.

- 마스킹 후 Agent가 "번호를 기억했다"고 답해도 실제 번호는 모델에 간 적이 없습니다.

  의도된 동작입니다. 마스킹과 기능성은 맞바꾸는 관계입니다.

확인: 마스킹 로그가 찍히고 요약 데모에서 대화가 길어질 때 메시지 개수가 줄어드는 것을 보면 성공입니다.

> 처리 목적에 따라 미들웨어가 자리 잡는 훅이 정해진다는 원리를 코드 리딩으로 확인했습니다.

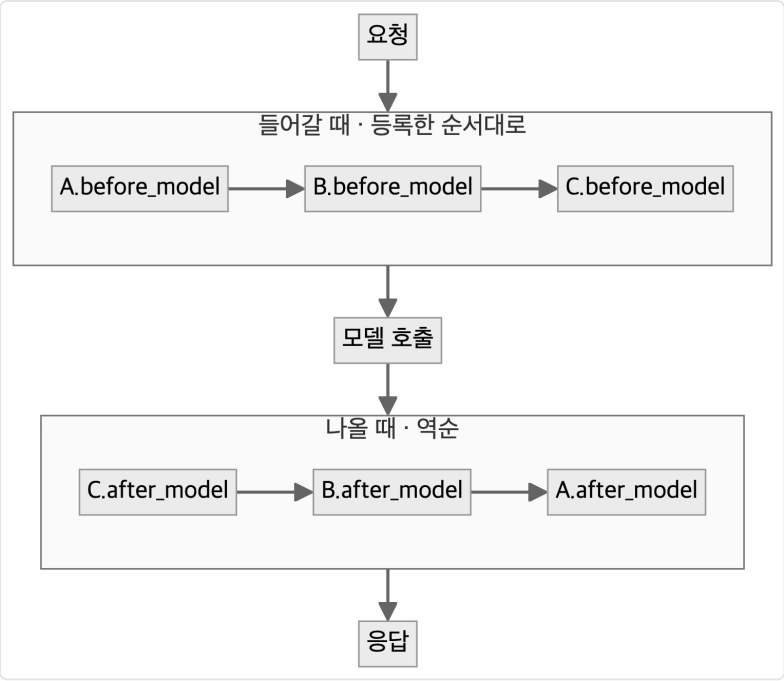
2-3. 미들웨어 조합 (실습)

> 여러 미들웨어를 겹칠 때 훅이 어떤 순서로 실행되는지 확인해봅니다.

미들웨어 여러 개를 등록하면 **양파(onion) 모델**로 실행됩니다.

요청이 양파 껍질을 밖에서 안으로 뚫고 들어갔다가 응답이 안에서 밖으로 나오는 그림입니다.

그래서 호출 순서가 훅마다 반대입니다. `before_model`은 등록 순서대로 불리고 `after_model`은 역순으로 불립니다.



그래서 배치 원칙은 "차단과 필터링은 앞쪽에, 로깅과 비용 집계는 뒤쪽에"입니다. 마스킹이 로깅보다 앞에 있어야 로그에도 마스킹된 내용이 남습니다.

따라하기

1. 순서 확인용 미들웨어를 `day5_practice/middleware_order.py`로 저장하고 실행 순서를 직접 관찰합니다.

```
# middleware_order.py - 미들웨어 조합의 실행 순서 관찰
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage

load_dotenv()


class TagMiddleware(AgentMiddleware):
    """순서 확인용: 이름표를 붙여 훅 호출 순서를 출력"""

    def __init__(self, tag: str):
        super().__init__()
        self.tag = tag

    @property
    def name(self) -> str:
        # 미들웨어는 이름으로 구분합니다. 같은 클래스를 여러 개 넣으려면 이름이 서로 달라야 합니
다
        return f"TagMiddleware_{self.tag}"

    def before_model(self, state, runtime):
        print(f"[{self.tag}] before_model")
        return None

    def after_model(self, state, runtime):
        print(f"[{self.tag}] after_model")
        return None


llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
agent = create_agent(
    model=llm,
    tools=[],
    middleware=[TagMiddleware("A"), TagMiddleware("B"), TagMiddleware("C")],
)
agent.invoke({"messages": [HumanMessage("안녕이라고만 답해줘")]})
```

2. 미들웨어 리스트의 순서를 `[TagMiddleware("C"), TagMiddleware("A"), TagMiddleware("B")]` 로 바꿔 다시 실행하고 출력 순서가 따라 바뀌는 것을 확인합니다.

실행 결과 예시

```
[A] before_model
[B] before_model
[C] before_model
[C] after_model
[B] after_model
[A] after_model
```

실전 조합 예시입니다. 순서에 의미가 있습니다. 2-1과 2-2에서 만든 클래스를 한 파일에 모으거나 import해서 씁니다.

```
agent = create_agent(
    model=llm,
    tools=[search_employee],
    middleware=[
        MaskingMiddleware(),        # 1순위: 개인정보를 먼저 가림
        SummarizationMiddleware(model=llm, max_tokens_before_summary=4000, mess
ages_to_keep=10),
        ToolCallLimitMiddleware(thread_limit=10),
        LoggingMiddleware(),        # 마지막: 정제된 최종 입력을 기록
    ],
)
```

막힐 때

- `AttributeError: property 'name' of 'TagMiddleware' object has no setter` 가 납니다. `AgentMiddleware` 가 `name` 을 읽기 전용으로 갖고 있어 하위 클래스에서 덮어쓸 수 없습니다. 이름표를 붙이려면 `self.tag` 처럼 다른 속성명을 씁니다.
- 로깅을 마지막에 넣었는데 after 로그가 제일 먼저 찍힙니다.

  정상입니다. `after_model` 은 역순으로 불립니다. 양파 모델을 다시 떠올립니다.
- 상태를 누적하는 미들웨어(예: 비용 집계)는 인스턴스를 변수로 분리해 둡니다.

  `middleware=[CostTracker()]` 처럼 익명으로 넣으면 호출이 끝난 뒤 누적값을 읽을 방법이 없습니다.

확인: before가 등록 순서대로 찍히고 after가 역순으로 찍히며 리스트 순서를 바꾸면 출력 순서도 바뀌면 성공입니다.

미들웨어는 양파 모델로 실행되므로 배치 순서 자체가 설계라는 걸 확인했습니다.